

UNDERACTUATED ROBOTICS

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-11-11.

[How to cite these notes, use annotations, and give feedback.](#)

Note: These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

CHAPTER 13

Robust and Stochastic Control

 Launch in Deepnote

My goal of presenting a relatively consumable survey of a few of the main ideas is perhaps more important in this chapter than any other. It's been said that "robust control is encrypted" (as in you need to know the secret code to get in). The culture in the robust control community has been to leverage high-powered mathematics, sometimes at the cost of offering more simple explanations. This is unfortunate, I think, because robotics and machine learning would benefit from a richer connection to these tools, and are perhaps destined to reinvent many of them.

The classic reference for robust control is [1]. [2] has a treatment that does more of its development in the time-domain and via Riccati equations; I will do the same here.

13.1 STOCHASTIC MODELS

So far in the notes, we have concerned ourselves primarily with known, deterministic systems. In the [stochastic systems](#) chapter, we started our study of nonlinear dynamics of stochastic systems, which can be beautiful! In this chapter we will begin to consider computational tools for analysis and control of those systems. Stochasticity can come in many forms... we may not know the governing equations (e.g. the coefficient of friction in the joints), our robot may be [walking on unknown terrain, subject to unknown disturbances](#), or even be picking up unknown objects. There are a number of mathematical frameworks for considering this uncertainty; for our purposes this chapter will generalize our thinking to equations of the form:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{w}, t) \quad \text{or} \quad \mathbf{x}[n+1] = \mathbf{f}(\mathbf{x}[n], \mathbf{u}[n], \mathbf{w}[n], n),$$

where $\mathbf{w}$ is a new *random* input signal to the equations capturing all of this potential variability. Although it is certainly possible to work in continuous time, and treat $\mathbf{w}(t)$ as a continuous-time random signal (c.f. [Wiener process](#)), the notation and intuition is a bit simpler when we work with $\mathbf{w}[n]$ as a discrete-time random signal. For this reason, we'll devote our attention in this chapter to the discrete-time systems.

In order to simulate equations of this form, or to design controllers against them, we need to define the random process that generates $\mathbf{w}[n]$. It is typical to assume the

values $\mathbf{w}[n]$ are independent and identically distributed (i.i.d.), meaning that $\mathbf{w}[i]$ and $\mathbf{w}[j]$ are uncorrelated when $i \neq j$. As a result, we typically define our distribution via a probability density $\mathbf{w}[n] \sim p_{\mathbf{w}}(\mathbf{w})$. This is not as limiting as it may sound... if we wish to treat temporally-correlated noise (e.g. "colored noise") the format of our equations is rich enough to support this by adding additional state variables; we'll give an example below of a "whitening filter" for modeling wind gusts. The other source of randomness that we will now consider in the equations is randomness in the initial conditions; we will similarly define a probability density $\mathbf{x}[0] \sim p_0(\mathbf{x})$.

The notation $\mathbf{w}[n] \sim p_{\mathbf{w}}(\mathbf{w})$ denotes "draw from the distribution $p_{\mathbf{w}}(\mathbf{w})$ ".

This modeling framework is rich enough for us to convey the key ideas; but it is not quite sufficient for all of the systems I am interested in. In **DRAKE** we go to additional lengths to support more general cases of **stochastic systems**. This includes modeling system parameters that are drawn from random each time the model is initialized, but are fixed over the duration of a simulation; it is possible but inefficient to model these as additional state variables that have no dynamics. In other problems, even the *dimension of the state vector* may change in different realizations of the problem! Consider, for instance, the case of a robot manipulating random numbers of dishes in a sink. I do not know many control formulations that handle this type of randomness well, and I consider this a top priority to think more about! (We'll begin to address it in the [output feedback chapter](#).)

13.2 COSTS AND CONSTRAINTS FOR STOCHASTIC SYSTEMS

Given a stochastic model, what sort of cost function should we write in order to capture the desired aspect of performance? There are a number of natural choices, which provide nice trade-offs between capturing the desired phenomena and computational tractability.

Remember that $\mathbf{x}[n]$ is now a random variable, so we want to write our cost and constraints using the distribution described e.g. $p_n(\mathbf{x})$. Broadly speaking, one might specify a cost in a handful of ways:

- **Desired state distribution.** One could specify a goal in terms of a desired state distribution, $p_n^d(\mathbf{x})$, and then try to minimize a measure of the difference between the actual and desired distributions.
- **Average cost,** $E[\sum_n \ell(\mathbf{x}[n], \mathbf{u}[n])]$, or some time-averaged/discounted variant. This is by far the most popular choice (especially in reinforcement learning), because it preserves the dynamic programming recursion for additive cost.
- **Worst-case cost,** e.g. $\min_{\mathbf{u}[\cdot]} \max_{\mathbf{x}[0], \mathbf{w}[\cdot]} \sum_n \ell(\mathbf{x}[n], \mathbf{u}[n])$. When we are able to upper-bound this cost and push down on that bound, then this is sometimes called *guaranteed cost control*.
- **Relative worst-case** (aka "gain bounds"). Even for linear systems, bounding the absolute performance (for all $\mathbf{w}$) leads to vacuous bounds; we can do much better by evaluating our performance metric relative to the magnitude of the disturbance input.
- **Value at Risk (VaR)** and other related metrics from economics and operations research. $\text{VaR}(\alpha)$ is the maximum cost if we ignore the tails of the distribution (worse outcomes whose combined probability is at most α). Note that VaR is easy to understand, but the related quantity "conditional value at risk" (CVaR) has superior mathematical properties, including connections to convex optimization [3].

- **Regret** formulations try to minimize the difference between the designed controller and e.g. an optimal controller in some class that has access to privileged information, such as knowing a priori the disturbances $\mathbf{w}[n]$. These formulations have become popular in the field of online optimization, and we are now starting to see "regret bounds" for control.

Similarly, we can think about how to generalize our deterministic constraints, e.g. $g(\mathbf{x}) \leq 0$, into a the stochastic framework:

- **Chance constraints**, which take the form e.g. $P(g(\mathbf{x}) > 0) \leq \alpha$. For instance, we might like to guarantee that the probability that our airplane will crash into a tree is less than α . Even for Gaussian uncertainty and polytopic constraints, these quantities can be hard to evaluate exactly; we often end up making approximations[4] though there is some hope to provide rigorous bounds with Lyapunov-like arguments [5].
- **Worst-case constraints**, which are the limit of the chance constraints when we take $\alpha \rightarrow 0$, tend to be much more amenable to computation, and connect directly to [reachability analysis](#).

The term "robust control" is typically associated with the class of techniques that try to guarantee some *worst-case* performance or a worst-case bound (e.g. the gain bounds). The term "stochastic optimal control" or "stochastic control" is typically used as a catch-all for other methods that reason about stochasticity, without necessarily providing the strict robustness guarantees.

It's important to realize that proponents of robust control are not necessarily pessimistic by nature; there is a philosophical stance about how difficult it can be to model the true distribution of randomness that a control system will face in the world. Worst-case analysis typically requires only a definition of the *set* of possible values that the random variable can take, and not a detailed *distribution* over those possible values. It may be more practical to operationalize a concept like "this UAV will not crash so long as the peak wind gusts are below 35 mph" than requiring a detailed distribution over wind gust probabilities. But this simpler specification does come at some cost -- worst-case certificates are often pessimistic about the true performance of a system, and optimizing worst-case performance can lead to conservatism in the control design.

13.3 FINITE MARKOV DECISION PROCESSES

We already had quick preview into stochastic optimal control in one of the cases where it is particularly easy: [finite Markov Decision Processes \(MDPs\)](#). And we have seen that [finite-state approximations](#) can give us insights into even very complex nonlinear dynamics. Let's make sure that we understand how to do control design again the various cost and constraint formulations we've proposed above...

13.4 LINEAR OPTIMAL CONTROL

13.4.1 Stochastic LQR

Let's consider a stochastic extension of the (discrete-time) LQR problem, where the system is now subjected to additive Gaussian white noise:

$$\begin{aligned}\mathbf{x}[n+1] &= \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n], \\ E[\mathbf{w}[i]] &= 0, \quad E[\mathbf{w}[i]\mathbf{w}^T[j]] = \delta_{ij}\mathbf{\Sigma}_{\mathbf{w}},\end{aligned}$$

where δ_{ij} is one if $i = j$ and zero otherwise, and $\Sigma_{\mathbf{w}}$ is the covariance matrix of the disturbance, and we take the average cost:

$$\min E \left[\sum_{n=0}^{\infty} \mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n] \right], \quad \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0.$$

Note that we saw one version of this already when we discussed [policy search for LQR](#).

In the standard LQR derivation, we started by assuming a quadratic form for the cost-to-go function. Is that a suitable starting place for this stochastic version? We've already [studied the dynamics of a linear system subjected to Gaussian noise](#), and learned that (at best) we should expect it to have a Gaussian stationary distribution. But that sounds problematic, no? If the system can not drive the system to zero and stay at zero in order to stop accruing cost, then won't the infinite-horizon cost be infinite (regardless of the controller)?

Yes. The cost-to-go for this problem is infinite! But it turns out that it still has enough structure for us to work with. In particular, let's "guess" a cost-to-go function of the form:

$$J_n(\mathbf{x}) = \mathbf{x}^T \mathbf{S}_n \mathbf{x} + c_n,$$

where c_n is a (scalar) constant. Now we can write the Bellman equation and do some algebra:

$$\begin{aligned} J_n(\mathbf{x}) &= \min_{\mathbf{u}} E_{\mathbf{w}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} + \mathbf{w}]^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} + \mathbf{w}] + c_{n+1}] \\ &= \min_{\mathbf{u}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}] + c_{n+1} \\ &\quad + E_{\mathbf{w}} [[\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]^T \mathbf{S}_{n+1} \mathbf{w} + \mathbf{w}^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]] + E_{\mathbf{w}} [\mathbf{w}^T \mathbf{w}]] \\ &= \min_{\mathbf{u}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}] + c_{n+1} + \text{tr}(\Sigma_{\mathbf{w}})]. \end{aligned}$$

The second line follows by simply pulling all of the deterministic terms outside of the expected value. The third line follows by observing that $\mathbf{x}$ and $\mathbf{u}$ are uncorrelated with $\mathbf{w}$, and $E[\mathbf{w}] = 0$, so those cross terms are zero in expectation. Notice that, apart from the c_{n+1} and $\Sigma_{\mathbf{w}}$, the remaining terms are exactly the same as the deterministic (discrete-time) LQR version.

Remarkably, we can therefore achieve our dynamic programming recursion by using $\mathbf{S}_n$ as the solution of the discrete Riccati equation, and $c_n = c_{n+1} + \text{tr}(\Sigma_{\mathbf{w}})$. As we take $n \rightarrow \infty$, $\mathbf{S}_n$ converges to the steady-state solution to the algebraic Riccati equation, and only c_n grows to infinity. As a result, even though the cost-to-go is infinite, the optimal control is still well defined: it is the same $\mathbf{u}^* = -\mathbf{K} \mathbf{x}$ that we obtain from the deterministic LQR problem!

Take note of the fact that the true cost-to-go blows up to infinity. In reinforcement learning, for instance, it is common practice to avoid this blow-up by considering discounted-cost formulations,

$$\sum_{n=0}^{\infty} \gamma^n \ell(\mathbf{x}[n], \mathbf{u}[n]), \quad 0 < \gamma \leq 1,$$

or average-cost formulations,

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]).$$

These are satisfactory solutions to the problem, but please make sure to understand why they *must* be used.

Non-i.i.d. disturbances

The LQR derivation above assumed that the disturbances $\mathbf{w}[n]$ were independent and identically distributed (i.i.d.). But many of the disturbances we would like to model are not i.i.d.. For instance, consider a UAV flying in the wind. The wind is correlated over time, sometimes building up to gusts but even those gusts are relatively long compared to any the sampling rate of a control system.

In fact, the standard models of wind are typically the output of a Gaussian i.i.d. random signal passed through a linear low-pass filter [6]. This suggests a very natural approach to using LQR to stabilize a UAV in the wind: design the controller for the joint wind + UAV system. The state of this system contains both the state of the UAV and the state of the wind, so the resulting controller will have the form

$$\mathbf{u} = -\mathbf{K} \begin{bmatrix} \mathbf{x}_{uav} \\ \mathbf{x}_{wind} \end{bmatrix}.$$

In practice, this means that the controller will need to rely on an online estimator for the wind state in addition to the standard (UAV) state estimator.

Example 13.1 (A quadrotor flying in the wind.)

I've adapted our simple 2D quadrotor model to have an (optional) additional input port for Cartesian force disturbances. For the wind, I've added a `RandomSource` to generate the output of iid Gaussain random process, multiplied that by a scalar `Gain` to scale the covariance, and then put it through a `FirstOrderLowPassFilter` to make the colored noise wind model.

In the notebook, I've implemented LQR both ways: first the traditional LQR on the quadrotor system *only* (ignoring the wind), and second on the diagram that includes the `Gain` and `LowPassFilter` systems in addition to the quadrotor.

 Launch in Deepnote

Stochastic linear MPC

Coming soon...

13.4.2 Worst-case control w/ bounded uncertainty

Common Lyapunov functions

We've already seen one nice example of robustness analysis for linear systems when we wrote a small optimization to find a [common Lyapunov function for uncertain linear systems](#). That example studied the dynamics $\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n]$ where the coefficients of $\mathbf{A}$ were unknown but bounded.

[We also saw](#) that essentially the same technique can be used to certify stability against disturbances, e.g.:

$$\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{w}[n], \quad \mathbf{w}[n] \in \mathcal{W},$$

where $\mathcal{W}$ describes some bounded set of possible uncertainties. In order to be compatible with convex optimization, we often choose to describe $\mathcal{W}$ as either an ellipsoid or as a convex polytope[7]. But let's remember a very important point: in this situation with additive disturbances, we can no longer expect the system to converge stably to the origin. Our example before used the common Lyapunov function only to certify the *invariance* of a region (if I start inside some region, then I'll never leave).

Polytope dynamics

A polytope is a bounded polyhedron. Let's take a moment to understand how a polytope propagates through a linear dynamical system. There are a number of possible representations for a polytope. The most common the so-called "H-rep", or half-space representations, which is described by the linear constraints $\mathbf{H}\mathbf{x} \leq \mathbf{h}$, or the "V-rep", or vertex representation, which is described by the convex hull of a set of vertices. For optimization, we often also study "zonotopes", which are defined as the affine projection of a unit cube, or the more general "AH-polytopes" which are the affine projection of an H-polytope [8, 9]. In Drake we have a [library of these convex sets](#) with many of the most common operations implemented and made compatible with `MathematicalProgram`.

Let's develop our notation for H-polytope and AH-polytopes here. We'll define a polytope, $\mathbb{P} \subset \mathbb{R}^n$, as an H-polytope using

$$\mathbb{P} = \mathcal{H}(\mathbf{H}, \mathbf{h}) := \{\mathbf{x} \in \mathbb{R}^n | \mathbf{H}\mathbf{x} \leq \mathbf{h}\}.$$

For an AH-polytope, we write

$$\mathbb{X} := \mathcal{AH}(\mathbf{T}, \mathbf{t}, \mathbb{P}) := \{\mathbf{T}\mathbf{x} + \mathbf{t} \in \mathbb{R}^n | \mathbf{x} \in \mathbb{P} \subset \mathbb{R}^m\},$$

where $\mathbf{T} \in \mathbb{R}^{n \times m}$ and $\mathbf{t} \in \mathbb{R}^n$ define the affine transform. Note that AH-polytope can be the projection of an H-polytope with a different ambient dimension.

AH-polytopes are closed under many of the essential operations we need for linear dynamical systems[9]. Using set notation, this includes (further) affine transformations:

$$\mathbf{A}\mathbb{X} + \mathbf{b} = \mathcal{AH}(\mathbf{A}\mathbf{T}, \mathbf{A}\mathbf{t} + \mathbf{b}, \mathbb{P}),$$

and the [Minkowski sum](#). Given two AH-polytopes, $\mathbb{X}$ and $\mathbb{Y}$ both in $\mathbb{R}^n$, their Minkowski sum is given by:

$$\mathbb{X} \oplus \mathbb{Y} = \mathcal{AH}\left([\mathbf{T}_x, \mathbf{T}_y], \mathbf{t}_x + \mathbf{t}_y, \mathcal{H}\left(\begin{bmatrix} \mathbf{H}_x & 0 \\ 0 & \mathbf{H}_y \end{bmatrix}, \begin{bmatrix} \mathbf{h}_x \\ \mathbf{h}_y \end{bmatrix}\right)\right).$$

We can actually use this set notation now directly to write the dynamics of a polytope (over state) as it propagates through an uncertain linear dynamical system:

$$\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{w}[n], \quad \mathbf{x}[0] \in \mathbb{X}_0, \mathbf{w}[n] \in \mathbb{W}.$$

The result is

$$\mathbb{X}_{n+1} = \mathbf{A}\mathbb{X}_n \oplus \mathbb{W}.$$

Beautiful! But please notice one important detail: the description size of the polytope (via the ambient dimension of the underlying H-polytope) is growing on each time step, due to the Minkowski sum with $\mathbb{W}$. The sets can get complicated for long-horizon rollouts.

Robust MPC

Although I've written out polytopic reachability for a closed-loop linear system (+ noise) so far, one can similarly propagate the polytopic dynamics under a feedback control of the form

$$\begin{aligned}\mathbf{x}[n+1] &= \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n], \\ \mathbb{X}_{n+1} &= (\mathbf{A}\mathbb{X}_n + \mathbf{B}\mathbf{u}_n) \oplus \mathbb{W}.\end{aligned}$$

This leads to a form of robust model-predictive control commonly referred to as "TubeMPC" [10]. One can optimize $\mathbf{u}_n$ and the descriptions of $\mathbb{X}_n$ using convex optimization subject to these linear dynamics. Furthermore, this can be combined with [disturbance-based feedback parameterizations](#) to simultaneously search for time-varying feedback gains, $\mathbf{K}_n$ [11].

Example 13.2 (Tube MPC)

Coming soon...

Polytopic containment

One objective we might like to specify in our optimization is that the entire reachable set at the final time is inside a goal set, e.g. $\mathbb{X}_N \subseteq \mathbb{X}_{goal}$. Checking this condition is a "polytopic containment" problem -- a classic problem in computational geometry. But we want to be able to optimize the form of $\mathbb{X}_N$ (via optimizing the sequence of control decisions), so we want to be able to embed the polytopic containment problem as just an additional set of constraints in our MPC problem.

[8] gave an efficient encoding of the polytopic containment problem using only linear constraints that is sufficient to certify the containment, and [9] gave an additional condition which is both necessary and sufficient, but requires potentially an exponential number of linear constraints. The specific equations are slightly messy, so I've omitted them here, but in these linear constraints, the terms $\mathbf{T}_x$, $\mathbf{t}_x$, $\mathbf{t}_{\hat{x}}$, and $\mathbf{h}_{\hat{x}}$ also appear linearly and so can be optimized simultaneously. Unfortunately, the terms $\mathbf{T}_{\hat{x}}$, $\mathbf{H}_x$, $\mathbf{h}_x$, and $\hat{\mathbf{H}}_{\hat{x}}$ appear bilinearly with the containment decision variables, so we typically treat them as fixed.

Example 13.3 (Polytopic containment)

Coming soon...

If one is simply performing reachability analysis (not joint with control design), then polytopic containment can also be used on each time step to limit the growth of the description length of the polytopes. We can attempt to replace $\mathbb{X}_n$ with a minimal volume polytope, $\hat{\mathbb{X}}_n$ with a fixed number of faces, subject to $\mathbb{X}_n \subseteq \hat{\mathbb{X}}_n$. This, of course, comes at the cost of conservatism (larger reachable sets).

Example 13.4 (TubeMPC with a goal polytope)

Robust constrained LQR

Coming soon...

13.4.3 Disturbance-based feedback parameterizations

Although it would be tempting to parameterize the feedback gains $\mathbf{K}_n$ as decision variables in our Robust MPC framework, searching for both $\mathbf{K}_n$ and $\mathbf{x}_n$ in the traditional parameterization of $\mathbf{K}_n \mathbf{x}_n$ would result in terms that are bilinear (and therefore nonconvex) in the decision variables. The disturbance-based feedback parameterizations get around this. This is an old but important idea which was made famous first as the [Youla parameterization](#) (alternatively "Q-parameterization"). In the time domain this typically leads to controllers which are "unrolled in time" and depend on a potentially infinite history of disturbances; common practice is to approximate these with a finite-impulse response (FIR) truncation. One could imagine extracting a state-space realization of these FIR responses using the techniques from [linear system identification](#) [12].

We can understand the essence of this idea with a simple extension of the [LQR with least-squares derivation](#)... (it's a work in progress!)

Given the state space equations:

$$\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n],$$

Consider parameterizing an output feedback policy of the form

$$\mathbf{u}[n] = \mathbf{K}_0[n]\mathbf{x}_0 + \sum_{i=1}^{n-1} \mathbf{K}_i[n]\mathbf{w}[n-i],$$

then the closed-loop state is convex in the control parameters, $\mathbf{K}$:

$$\mathbf{x}[n] = \left(\mathbf{A}^n + \sum_{i=0}^{n-1} \mathbf{A}^{n-i-1} \mathbf{B} \mathbf{K}_0[i] \right) \mathbf{x}_0 + \sum_{j=0}^{n-1} \sum_{i=0}^{n-1} \mathbf{A}^{n-i-1} \mathbf{B} \mathbf{K}_j[i] \mathbf{w}[i-j],$$

and therefore objectives that are convex in $\mathbf{x}$ and $\mathbf{u}$ (like LQR) are also convex in $\mathbf{K}$. Moreover, we can calculate $\mathbf{w}[n]$ by the time that it is needed given our observations of $\mathbf{x}[n+1]$, $\mathbf{x}[n]$, and knowledge of $\mathbf{u}[n]$.

13.4.4 L_2 gain

By limiting $\mathbf{w}$ to be drawn from a bounded (polytopic) set, we were able to perform efficient reachability analysis. But knowing that the dependency on $\mathbf{w}$ is linear allows us to do something potentially even more natural, which can lead to tighter bounds. In particular, we expect the *magnitude* of the deviation in $\mathbf{x}$ compared to the undisturbed case to be proportional to the *magnitude* of the disturbance, $\mathbf{w}$. So a perhaps more natural bound for a linear system is a *relative* bound on the magnitude of the response (from zero initial conditions) relative to the magnitude of the disturbance.

Typically, this is done with the a scalar " L_2 gain", γ , defined as:

$$\begin{aligned} \operatorname{argmin}_{\gamma} \quad \text{subject to} \quad & \sup_{\mathbf{w}(\cdot) \in \int \|\mathbf{w}(t)\|^2 dt \leq \infty} \frac{\int_0^T \|\mathbf{x}(t)\|^2 dt}{\int_0^T \|\mathbf{w}(t)\|^2 dt} \leq \gamma^2, \quad \text{or} \\ \operatorname{argmin}_{\gamma} \quad \text{subject to} \quad & \sup_{\mathbf{w}[\cdot] \in \sum_n \|\mathbf{w}[n]\|^2 \leq \infty} \frac{\sum_0^N \|\mathbf{x}[n]\|^2}{\sum_0^N \|\mathbf{w}[n]\|^2} \leq \gamma^2. \end{aligned}$$

The name " L_2 gain" comes from the use of the ℓ_2 norm on the signals $\mathbf{w}(t)$ and $\mathbf{x}(t)$, which is assumed only to be finite.

More often, these gains are written not in terms of $\mathbf{x}[n]$ directly, but in terms of some "performance output", $\mathbf{z}[n]$. For instance, if we would like to bound the cost of a quadratic regulator objective as a function of the magnitude of the disturbance, we can minimize

$$\min_{\gamma} \quad \text{subject to} \quad \sup_{\mathbf{w}[n]} \frac{\sum_0^N \|\mathbf{z}[n]\|^2}{\sum_0^N \|\mathbf{w}[n]\|^2} \leq \gamma^2, \quad \mathbf{z}[n] = \begin{bmatrix} \sqrt{\mathbf{Q}}\mathbf{x}[n] \\ \sqrt{\mathbf{R}}\mathbf{u}[n] \end{bmatrix}.$$

This is a simple but important idea, and understanding it is the key to understanding the language around robust control. In particular the $\mathcal{H}_2$ norm of a system (from input $\mathbf{w}$ to output $\mathbf{z}$) is the energy of the impulse response; when $\mathbf{z}$ is chosen to represent the quadratic regulator cost as above, it corresponds to the expected LQR cost. The $\mathcal{H}_{\infty}$ norm of a system (from $\mathbf{w}$ to $\mathbf{z}$) is the largest singular value of the transfer function; it corresponds to the L_2 gain.

Dissipation inequalities

One of the mechanisms for certifying an L_2 gain for a system comes from a generalization of Lyapunov analysis to consider the contributions of system inputs via the so-called "dissipation inequalities". Dissipation inequalities are a general tool, and L_2 -gain analysis is only one application of them; for a broader treatment see [13] or [14, Ch. 2].

Informally, the idea is to generalize the Lyapunov conditions, $V(\mathbf{x}) \succ 0$, $\dot{V}(\mathbf{x}) \preceq 0$, into the more general form

$$V(\mathbf{x}) \succ 0, \quad \dot{V}(\mathbf{x}) \preceq s(\mathbf{x}, \mathbf{w}),$$

where $\mathbf{w}$ is the input of interest in this setting, and $s()$ is a scalar quantity representing a "supply rate". Once a system has input, the value of V may go up or it may go down, but if we can bound the way that it goes up by a simple function of the input, then we may still be able to provide input-to-state or input-output bounds for the system. Integrating both sides of the derivative condition with respect to time yields:

$$\forall \mathbf{x}(0), \quad V(\mathbf{x}(T)) \leq V(\mathbf{x}(0)) + \int_0^T s(\mathbf{x}(t), \mathbf{w}(t)) dt.$$

To obtain a bound on the L_2 gain between input $\mathbf{w}(t)$ and output $\mathbf{z}(t)$, the supply rate of interest is

$$s(\mathbf{x}, \mathbf{w}) = \gamma^2 \|\mathbf{w}\|^2 - \|\mathbf{z}\|^2,$$

which yields

$$\forall \mathbf{x}(0), \quad V(\mathbf{x}(T)) \leq V(\mathbf{x}(0)) + \int_0^T \gamma^2 \|\mathbf{w}(t)\|^2 dt - \int_0^T \|\mathbf{z}(t)\|^2 dt.$$

Now, since this must hold for all $\mathbf{x}(0)$, it holds for $\mathbf{x}(0) = 0$. Furthermore, we know $V(\mathbf{x}(T))$ is non-negative, so we also have

$$0 \leq V(\mathbf{x}(T)) \leq \int_0^T \gamma^2 \|\mathbf{w}(t)\|^2 dt - \int_0^T \|\mathbf{z}(t)\|^2 dt.$$

Therefore, if we can find a V that satisfied the dissipation inequality for this storage function, we have certified the γ is an L_2 gain for the system:

$$\frac{\int_0^T \|\mathbf{z}(t)\|^2 dt}{\int_0^T \|\mathbf{w}(t)\|^2 dt} \leq \gamma^2.$$

Small-gain theorem

Coming soon...

Model uncertainty as a special case.

13.4.5 Robust LQR as $\mathcal{H}_\infty$

Let's consider a robust variant of the LQR problem:

$$\begin{aligned} \min_{\mathbf{u}[\cdot]} \max_{\mathbf{w}[\cdot]} \sum_{n=0}^{\infty} & \mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n] - \gamma^2 \mathbf{w}^T[n] \mathbf{w}[n], \\ & \mathbf{x}[n+1] = \mathbf{A} \mathbf{x}[n] + \mathbf{B} \mathbf{u}[n] + \mathbf{B}_w \mathbf{w}[n], \\ & \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0. \end{aligned}$$

The reason for this choice of cost function will become clear in the derivation, but the intuition is that we want to reward the controller for having a small response to large $\mathbf{w}[\cdot]$. Note that unlike the stochastic LQR formulation, here we do not specify the distribution over $\mathbf{w}[n]$, and in fact we don't even restrict it to a bounded set. All we know is that at each time step, an omniscient adversary is allowed to choose the $\mathbf{w}[n]$ that tries to maximize this objective.

In fact, since we don't need to specify a continuous-time random process and the continuous-time derivation is both cleaner and by now, I think, more familiar, let's do this one in continuous time.

$$\begin{aligned} \min_{\mathbf{u}[\cdot]} \max_{\mathbf{w}[\cdot]} \int_{n=0}^{\infty} & dt [\mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) + \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t) - \gamma^2 \mathbf{w}^T(t) \mathbf{w}(t)], \\ & \dot{\mathbf{x}}(t) = \mathbf{A} \mathbf{x}(t) + \mathbf{B} \mathbf{u}(t) + \mathbf{B}_w \mathbf{w}(t), \\ & \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0. \end{aligned}$$

We will once again guess a quadratic cost-to-go function:

$$J(\mathbf{x}) = \mathbf{x}^T \mathbf{S} \mathbf{x}, \quad \mathbf{S} = \mathbf{S}^T \succ 0.$$

The dynamic programming recursion still holds for this problem, resulting in the Bellman equation:

$$\forall \mathbf{x}, \quad 0 = \min_{\mathbf{u}} \max_{\mathbf{w}} \left[\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} - \gamma^2 \mathbf{w}^T \mathbf{w} + \frac{\partial J^*}{\partial \mathbf{x}} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} + \mathbf{B}_w \mathbf{w}] \right].$$

Since the inside is a concave quadratic form over $\mathbf{w}$, we can solve for the adversary by finding the maximum:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} &= -2\gamma^2 \mathbf{w}^T + 2\mathbf{x}^T \mathbf{S} \mathbf{B}_w, \\ \mathbf{w}^* &= \frac{1}{\gamma^2} \mathbf{B}_w^T \mathbf{S} \mathbf{x}. \end{aligned}$$

This is the disturbance input that can cause the largest cost; the optimal play for the adversary. Now we can solve the remainder as we did with the original LQR problem:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{u}} &= 2\mathbf{u}^T \mathbf{R} + 2\mathbf{x}^T \mathbf{S} \mathbf{B}, \\ \mathbf{u}^* &= -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} \mathbf{x} = -\mathbf{K} \mathbf{x}. \end{aligned}$$

Substituting this back into the Bellman equation gives:

$$0 = \mathbf{Q} + \mathbf{S} [\gamma^{-2} \mathbf{B}_w \mathbf{B}_w^T - \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T] \mathbf{S} + \mathbf{S}^T \mathbf{A} + \mathbf{A}^T \mathbf{S},$$

which is the original LQR Riccati equation with one additional term involving γ . And like the original LQR Riccati equation, we must ask whether it has a positive-definite solution for $\mathbf{S}$. It turns out that if the system is stabilizable and γ large enough, then it does have a PD solution. However, as we reduce γ towards zero, there will be some threshold γ beneath which this Riccati equation does not have a PD solution. Intuitively, if γ is too small, then the adversary is rewarded for injecting disturbances that are so large as to break the convergence.

Here is the fascinating thing: the γ in this robust LQR cost function can be interpreted as an L_2 gain for the system. Recall that when we were making connections between the Bellman equation and Lyapunov equations, we observed that the Bellman equation can be written as $\forall \mathbf{x}, \quad \dot{J}^*(\mathbf{x}) \leq -\ell(\mathbf{x}, \mathbf{u})$? Here this yields:

$$\forall \mathbf{x}, \quad \dot{J}^*(\mathbf{x}) \leq \gamma^2 \mathbf{w}^T(t) \mathbf{w}(t) - \mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) - \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t).$$

This means that the cost-to-go is a valid dissipation inequality for the supply rate that provides an L_2 gain for the performance output $\mathbf{z} = \begin{bmatrix} \sqrt{\mathbf{Q}} \mathbf{x} \\ \sqrt{\mathbf{R}} \mathbf{u} \end{bmatrix}$. Moreover, we can find

the minimal L_2 gain by finding the minimal $\gamma > 0$ for which the Riccati equation has a positive-definite solution. And given the properties of the Riccati equation, this can be done with a line search in γ .

Because the L_2 gain is the $\mathcal{H}_\infty$ -norm of the system, this recipe of searching for the smallest γ and taking the Riccati solution is an instance of $\mathcal{H}_\infty$ control design.

13.4.6 Linear Exponential-Quadratic Gaussian (LEQG)

[15] observed that it is also straight-forward to minimize the objective:

$$J = E \left[\prod_{n=0}^{\infty} e^{\mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n]} e^{\mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n]} \right] = E \left[e^{\sum_{n=0}^{\infty} \mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n]} \right],$$

with $\mathbf{Q} = \mathbf{Q}^T \geq \mathbf{0}$, $\mathbf{R} = \mathbf{R}^T > 0$, by observing that the cost is monotonically related to $\log J$, and therefore has the same minima (this same trick forms the basis for "geometric programming" [16]). This is known as the "Linear Exponential-Quadratic Gaussian" (LEG or LEQG), and for the deterministic version of the problem (no process nor measurement noise) the solution is identical to the LQR problem; it adds no new modeling power. But with noise, the LEQG optimal controllers are different from the LQG controllers; they depend explicitly on the covariance of the noise.

[17] provides an extensive treatment of this framework; nearly all of the analysis from LQR/LQG (including Riccati equations, Hamiltonian formulations, etc) have analogs for their versions with exponential cost, and he argues that LQG and H-infinity control can (should?) be understood as special cases of this approach.

13.4.7 Adaptive control

The standard criticism of $\mathcal{H}_2$ optimal control is that minimizing the expected value does not allow any guarantees on performance. The standard criticism of $\mathcal{H}_\infty$ optimal control is that it concerns itself with the worst case, and may therefore be conservative, especially because distributions over possible disturbances chosen a priori may be unnecessarily conservative. One might hope that we could get some of this performance back if we are able to update our models of uncertainty online, adapting to the statistics of the disturbances we actually receive. This is one of the goals of adaptive control.

One of the fundamental problems in online adaptive control is the trade-off between exploration and exploitation. Some inputs might drive the system to build more accurate models of the dynamics / uncertainty quickly, which could lead to better performance. But how can we formalize this trade-off?

There has been some nice progress on this challenge in machine learning in the setting of (contextual) [multi-armed bandit](#) problems. For our purposes, you can think of bandits as a limiting case of an optimal control problem where there are no dynamics (the effects of one control action do not effect the results of the next control action). In this simpler setting, the online optimization community has developed exploration-exploitation strategies based on the notion of minimizing *regret* -- typically the accumulated difference in the performance achieved by my online algorithm vs the performance that would have been achieved if I had been privy to the data from my experiments before I started. This has led to methods that make use of concepts like upper-confidence bound (UCB) and more recently bounds using a least-squares squares confidence bound [18] to provide bounds on the regret.

In the last few years, we've see these results translated into the setting of linear optimal control...

13.4.8 Structured uncertainty

13.4.9 Linear parameter-varying (LPV) control

13.5 TRAJECTORY OPTIMIZATION

13.5.1 Monte-carlo trajectory optimization

13.5.2 Iterative $\mathcal{H}_2$ /iLQG

13.6 NONLINEAR ANALYSIS AND CONTROL

13.7 DOMAIN RANDOMIZATION

13.8 EXTENSIONS

13.8.1 Alternative risk/robustness metrics

REFERENCES

1. Kemin Zhou and John C. Doyle, "Essentials of Robust Control", Prentice Hall , 1997.
2. I. R. Petersen and V. A. Ugrinovskii and A. V. Savkin, "Robust Control Design using H-infinity Methods", Springer-Verlag , 2000.
3. Anirudha Majumdar and Marco Pavone, "How should a robot assess risk? towards an axiomatic theory of risk in robotics", *Robotics Research* , pp. 75--84, 2020.
4. Guy Scher and Sadra Sadraddini and Russ Tedrake and Hadas Kress-Gazit, "Elliptical Slice Sampling for Probabilistic Verification of Stochastic Systems with Signal Temporal Logic Specifications", *To appear in the Proceedings of Hybrid Systems: Computation and Control (HSCC)* , 2022. [[link](#)]
5. Jacob Steinhardt and Russ Tedrake, "Finite-time Regional Verification of Stochastic Nonlinear Systems", *International Journal of Robotics Research*, vol. 31, no. 7, pp. 901-923, June, 2012. [[link](#)]
6. Joseph Moore, "Robust Post-Stall Perching with a Fixed-Wing UAV", PhD thesis, Massachusetts Institute of Technology, September, 2014. [[link](#)]
7. Stephen Boyd and Lieven Vandenbergh, "Convex Optimization", Cambridge University Press , 2004.
8. Sadra Sadraddini and Russ Tedrake, "Linear Encodings for Polytope Containment Problems", *Proceedings of the 2019 IEEE 58th Conference on Decision and Control (CDC)* , pp. 8, 2019. [[link](#)]
9. Sadra Sadraddini and Russ Tedrake, "Linear Encodings for Polytope Containment Problems: Theoretical Foundations and Applications", , Feb, 2021.
10. D.Q. Mayne and M.M. Seron and SV Rakovic, "Robust model predictive control of constrained linear systems with bounded disturbances", *Automatica*, vol. 41, no. 2, pp. 219--224, 2005.
11. Sadra Sadraddini and Russ Tedrake, "Robust Output Feedback Control with Guaranteed Constraint Satisfaction", *In the Proceedings of 23rd ACM International Conference on Hybrid Systems: Computation and Control* , pp. 12, April, 2020. [[link](#)]
12. James Anderson and Nikolai Matni, "Structured state space realizations for SLS distributed controllers", *2017 55th Annual Allerton Conference on Communication, Control, and Computing (Allerton)* , pp. 982-987, 2017.
13. Christian Ebenbauer and Tobias Raff and Frank Allgower, "Dissipation inequalities in systems theory: An introduction and recent results", *Invited*

14. Carsten Scherer and Siep Weiland, "Linear Matrix Inequalities in Control", Online Draft , pp. 293, 2015.
15. D. Jacobson, "Optimal stochastic linear systems with exponential performance criteria and their relation to deterministic differential games", *IEEE Transactions on Automatic Control*, vol. 18, no. 2, pp. 124--131, apr, 1973.
16. S. Boyd and S.-J. Kim and L. Vandenberghe and A. Hassibi, "A Tutorial on Geometric Programming", *Optimization and Engineering*, vol. 8, no. 1, pp. 67-127, 2007.
17. Peter Whittle, "Risk-sensitive optimal control", Wiley New York , vol. 20, 1990.
18. Dylan Foster and Alexander Rakhlin, "Beyond UCB: Optimal and Efficient Contextual Bandits with Regression Oracles", *Proceedings of the 37th International Conference on Machine Learning* , vol. 119, pp. 3199--3210, 13--18 Jul, 2020.
19. Sergey Levine and Chelsea Finn and Trevor Darrell and Pieter Abbeel, "End-to-end training of deep visuomotor policies", *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1334--1373, 2016.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

